

Project B - Semester 2, 2026

Table of contents

1	Univariate Polynomials with Integer Coefficients	1
1.1	Setting up your GitHub repo for hand-in (10pts):	2
2	Tasks (90pts)	3
2.1	Task 1 (15pts) - Working in an existing codebase	3
2.1.1	Study the Repo	3
2.1.2	Create Example Scripts (7 points)	3
2.1.3	Pretty Printing (8 points)	3
2.2	Task 2 (20pts) - Utilising Julia's type system	4
2.2.1	Showing overflow for PolynomialDense (2 points)	4
2.3	Task 3 (15pts) - Refactoring the data structure used	4
2.3.1	Sparse Polynomials	4
2.3.2	Constructors (2 points)	4
2.3.3	Operations (9 points)	4
2.3.4	Checking it works (4 points)	5
2.4	Task 4 (TODOpts) - Optimising Algorithms	5
2.4.1	Fast Powers (TODO points)	5
2.4.2	Fast Evaluation (TODO points)	6
2.4.3	Johnson's Algorithm (TODO points)	6
2.5	Task 5 (8pts) - Implementing $\mathbb{Z}_p$	6
2.5.1	Constructors (2 points)	6
2.5.2	Field operations and miscellaneous functions (4 points)	7
2.5.3	Checking that it works (2 points)	8
2.6	Task 6 (7pts) - Using $\mathbb{Z}_p[x]$	8
2.6.1	Constructing $\mathbb{Z}_p[x]$ (5 points)	8

1 Univariate Polynomials with Integer Coefficients

(last edit: Sep 18, 2026)

Individual work: This is an individual work assignment. Plagiarism will *not* be accepted. Nevertheless, feel free to consult with friends or classmates via Ed and other means about how to go about certain tasks. Note that similar and not identical projects were issued in previous editions of the course. If you wish, you may look at GitHub repos of such projects from previous years (in case students made them public). Do not copy code from those repos directly - but rather use such code bases for help as needed.

Note: The teaching staff will only answer questions (via Ed, consultation hour, or practicals) regarding this assignment up to late Wednesday evening before the due date.

Weights and marking criteria: Total number of points: 100.

There are 10 points for setting up the GitHub repo properly and following instructions for hand-in (including voice recording). The remaining 90 points are for the project tasks.

Submission format: This assignment should be submitted via a GitHub Repo and a PDF file via Blackboard.

Specific instructions for the GitHub repo are below. It is important that the GitHub repo be **private** and created within the MATH25042026 github organization. It is also important to name the repo exactly as PROJECTB-2504-2026-`<<FIRST NAME>>-<<LAST NAME>>`. So for example if your name is “Pierre-Simon Laplace”, the repo name should be PROJECTB-2504-2026-Pierre-Simon-Laplace.

The PDF must have the following components in this order:

1. Your name, student number, and assignment title (Project B - MATH2504 - 2026) on the top.
2. A (clickable) link to your GitHub repo.
3. A screenshot or copy/paste of output for each `example_script_task_i.jl`.
4. A screenshot or copy/paste the output of `test/runtests.jl` for the relevant tasks.

Note: *organize your submission into clearly marked sections. Use headings in the format `Task i` or `Task i.j` for each part of the assignment.*

Present the new snippets of code that you created and indicate what they do via brief descriptions. It is also a place to answer any questions that require output, or description.

The recommended way to make the PDF file is via a Jupyter notebook where you copy in some code and output into the notebook, and in certain cases use `include()` to run Julia code if appropriate. Also comment on questions in this PDF (e.g. when asked to answer things not via code). If desired keep this jupyter notebook (`.ipynb` file) in the repo. However, this Jupyter notebook will not be checked (only the PDF file), and it is not required to be a “runnable” notebook. In any case, please avoid pixilated screenshots of code ([this page](#) may be useful), so for example if you choose to format your PDF file in MS-word (instead of Jupyter), make sure the code is clean, formatted, and readable.

Feedback will be given by the teaching staff by annotating **selected parts** of your PDF file via blackboard. The teaching staff will also inspect your GitHub repo. A very readable and clean PDF file is important. Note that in printing Jupyter to PDF (or exporting to PDF) there may sometimes be excessive white space. Do not worry about this extra white space; it is not a problem.

Note: *long lines of code in Jupyter notebooks may be truncated by the width of the page when converted to a PDF. Please be aware of this and format your notebook (i.e., split code over multiple lines if needed) as appropriate.*

Marking Criteria:

- *10 points* are allocated for setting up GitHub according to the instructions and following the submission instructions.
- *90 points* are allocated for the polynomial project itself. Points are awarded for completing each of the tasks with full marks given to clean readable code.
- In general, points **will be** deducted for sloppy coding style. Make sure your code is properly indented, uses sensible and consistent variable names and write code that is in general clean and consistent.

TODO - something about put test results in final submission once, rather than reporting test results separately. Maybe still via screenshots but they should have their own section. Bit more description about the report. Some more intro to type system, how to get started, how to use `Revise.jl/pkg> dev .` and so on.

1.1 Setting up your GitHub repo for hand-in (10pts):

- Ideally use the same account you used for BigHW and Project A.
- **Duplicate** the repository https://github.com/dietmaroelz/2504_2026_projectB as described [here](#). You will first create your new repository and then follow the 4 steps under “Mirroring a repository”.

*Ideally, you would fork the repository, but GitHub does not allow forked repositories to be made private. Do **not** attempt to fork the repository.*

- Invite the course GitHub user, `MATH2504-staff-2026` as a collaborator and verify this is done (*failure to do so may result in a grade of 0*). You must do this no later than the project due date. It is your duty to check that we accepted the invite (give us 2-5 working days to accept the invite).
- Do **not** make any changes (i.e., commits) to the repo after the project due date.
- Create a local clone of the repo. It is *recommended* that you use the `git` command line to make changes/additions to the assignment and submission. However you are free to use any other mechanism (VS-Code, GitHub desktop, etc). You should make intermediate commits to demonstrate your development process.

Your GitHub repo must be formatted like the original repo.

- Ensure that there aren't excessive files in your submission repo. An exception is perhaps a Jupyter `.ipynb` file if you choose to use it for creating the PDF. Use the `.gitignore` file to ensure `git` does not commit additional files and output files to your repo.

2 Tasks (90pts)

Build your project on top of the [repo](#) by making modifications, additions, and improvements. Your first goal is to understand the base repository. After that, you will make improvements, additions, and cleanups.

Follow the supplied type names below. These are given so that tasks have backwards compatibility. Note that in an actual project, you would sometimes replace older types with newer ones.

Read all the tasks before starting - only some tasks are dependent on previous ones so they do *not* necessarily have to be done in order.

2.1 Task 1 (15pts) - Working in an existing codebase

This task is for getting the repo to run, improving pretty printing, and creating your own example script.

2.1.1 Study the Repo

Once you create your own repository, first read the file `README.md`. Then make sure the repository works by running `test/runtests.jl` and `example_script.jl`. At this point simply study the functionality of the repository by looking at the function signatures (not implementations) to understand what the repo can do.

2.1.2 Create Example Scripts (7 points)

Create your own example script called `example_script_task_1.jl` and include polynomials for illustrating the key functionality of the operations available via the software. Your script should run and demonstrate (using `println` or `@show`) how (*not why*) the code in the repository works. Namely:

1. construct (at least) *three* polynomials (say `f`, `g`, and `h`) using the given constructors for `PolynomialDense`,
2. compute `f + g`, `f * h`, `g * h`,
3. compute `derivative(f * g)` and show it is equal to what is expected when utilizing product rule,
4. compute `(f * h) ÷ h` modulo 5, 17 and 101, and confirm using `mod` (in `polynomial_dense.jl`) that `f` modulo 5, 17, and 101 is returned.
5. compute `gcd_mod_p(f*h, g*h, p)` for `p` equal to 5, 11 and 13.

Basically, `example_script_task_1.jl` is an improvement over the minimally supplied `example_script.jl`.

2.1.3 Pretty Printing (8 points)

Improve the “pretty printing” of the polynomials by incorporating the following rules:

- Terms of zero degree (i.e., constants) should exclude x^0 , e.g. 2 not $2x^0$.
- Terms with unit degree should exclude the degree, e.g. $3x$ not $3x^1$.
- Excluding the constant term 1, terms with unit coefficient should exclude the coefficient, e.g. $3x^2 + x$ not $3x^2 + 1x$.
- Use subtraction instead of adding negative terms, e.g. $3x^2 - 2x$ not $3x^2 + -2x$.
- Display the terms in *descending* degree order. $a_n \cdot x^n + a_{n-1} \cdot x^{n-1} + \dots + a_1 x + a_0$.
- Do not print terms with zero coefficients, e.g. $x^2 + 1$ not $x^2 + 0 x^1 + 1$. *Note: feel free to use Unicode characters (but this is not necessary).*
- Include the output of `example_script_task_1.jl` in your PDF submission via screenshot (or copy/paste).

2.2 Task 2 (20pts) - Utilising Julia's type system

In this task we parameterise the coefficient type to utilize `BigInt`

2.2.1 Showing overflow for PolynomialDense (2 points)

Exact computation requires polynomials with (truly) integer coefficients and thereby we must use `BigInt` rather than (the finitely many integers comprising) `Int`.

- Create a new script called `example_script_task_2.jl`. Demonstrate using five hard-coded examples (i.e, explicitly write out the polynomials and do not use `rand` to generate them) of `PolynomialDense` addition that will overflow by making the coefficients (*not the degree*) excessively large.
- TODO - Replace with optimising current polynomial dense addition with a vectorised version (the current one is very slow)

2.3 Task 3 (15pts) - Refactoring the data structure used

2.3.1 Sparse Polynomials

The current implementation for `PolynomialDense` uses a `Vector` to store the term of degree k at position $k+1$. This is optimal for dense polynomials (those with mostly *nonzero* coefficients) but our factoring algorithms rely heavily on manipulating *sparse polynomials* like $x^n - 1$. In this task we address this deficiency:

2.3.2 Constructors (2 points)

- Navigate to the directory `src/polynomial_definitions` and duplicate `polynomial_dense.jl` as `polynomial_sparse.jl`.
- Inside `src/polynomial_sparse.jl` rename all instances of `PolynomialDense` to `PolynomialSparse` and modify the inner constructor of `PolynomialSparse` so that *zero terms* are excluded from storage. Utilize the `max heap` (which you can find in `src/Utils/heap.jl`) ordered by term degree to accomplish this.
- Rewrite the *constructors* in `src/polynomial_sparse.jl` to account for the new underlying representation.

2.3.3 Operations (9 points)

- Rewrite the *functions* in `src/polynomial_sparse.jl` to account for the new underlying representation. For example, `mod` can no longer utilise array indexing.
- Navigate to the directory `src/basic_polynomial_operations` and duplicate the directory `dense` as the directory `sparse`.
- Refactor the *operations* in `src/basic_polynomial_operations/sparse` to account for the new underlying representation. For example, `+` can no longer be done with simple vector addition. To optimise these you may also need to refactor functions in `src/polynomial_sparse.jl`.

*Note: sparse term multiplication and addition can be optimised significantly compared to the abstract implementations. You should override both these functions for your `PolynomialSparse` implementation. You cannot achieve full marks in this section without optimisation to **both** addition and term multiplication. You may need to optimise other helper functions in order to achieve this.*

2.3.4 Checking it works (4 points)

7. Replicate the `example_script_task_2.jl` for `PolynomialSparse` in a file called `example_script_task_3.jl`.
8. Show (using `BenchmarkTools`) that there are examples where `PolynomialSparse` outperforms `PolynomialDense` for both time and memory usage. *Do this only for cases where `PolynomialSparse` and `PolynomialDense` do not overflow.*
9. Append your code for task 3.8 to `example_script_task_3.jl` and include the output of the script in your PDF submission.
10. Describe (at least) three pros and/or cons (in a table within your PDF) between the dense and sparse representations.

2.4 Task 4 (TODOpts) - Optimising Algorithms

Many of the algorithms provided within the codebase are not optimised. We (you) will improve several of these. **For each of the algorithms described below, complete the following tasks** (note - read in full the relevant algorithm description on what is considered sufficient testing/benchmarking and which functions are to be optimised before starting the sub-task):

1. (PDF Submission) Benchmark the existing implementation using `BenchmarkTools` and report the results.
2. Refactor all specified functions into the more efficient algorithm.
3. Create tests for your new optimised implementation to check it's correctness. All tests for this tasks should be placed in the file `test/algorithms_test` (you will need to create this file). You may choose which `Polynomial` concrete subtype to run your tests on. It is recommended you write additional tests beyond the minimum requirements below to ensure your implementation is correct (you will *not* be marked on these additional tests, but you will be marked on the correctness of your implementation).
4. (PDF Submission) Benchmark your new implementation using `BenchmarkTools` and report the results.
5. (PDF Submission) State the time complexity of the initial implementation and your new optimised implementation.

2.4.1 Fast Powers (TODO points)

As discussed in class, if you have a polynomial f and raise it to the power 2^n then instead of doing about 2^n multiplications you can do about n using **repeated squaring**. E.g., to compute f^8 , we compute $f^2 = f \cdot f$, then $f^4 = f^2 \cdot f^2$ and finally $f^8 = f^4 \cdot f^4$. This idea can be extended to computing f^n for any $n \in \mathbb{N}$! Utilise this algorithm to optimise the existing implementation of $\wedge$. You may wish to short-circuit computations of f^0 and f^1 (this is not required). This algorithm should be implemented at the abstract level (i.e., `Polynomial{C}`).

For benchmarking, find a single instance where the existing implementation takes on average at least 10s to run. Demonstrate that your implementation is indeed an optimisation.

Test your implementation of $\wedge$ for at least 10^2 polynomials (your tests should not take more than 30s to run on your computer). To construct polynomials where you can easily check the correctness of the result, use the [Binomial Theorem](#) alongside polynomials $(1 + x)^n$ for various n .

2.4.2 Fast Evaluation (*TODO points*)

Our current evaluation strategy for a polynomial $a_0 + a_1 \cdot x + \dots + a_n \cdot x^n$ is for the i 'th term take x to the power i , then multiply by the i 'th coefficient a_i . This however, is inefficient - [Horner's method](#) offers a much faster evaluation method, by implicitly utilising the following representation. Utilise this algorithm to optimise the existing implementation of `evaluate`. This algorithm should be implemented at the abstract level, but should be optimised for both sparse/dense representations simultaneously (*hint:*)

For benchmarking, find a single instance where the existing implementation takes on average at least 5s to run. Demonstrate that your implementation is indeed an optimisation.

Test your implementation of `evaluate` for at least 10^2 random polynomials (your tests should not take more than 30s to run on your computer).

2.4.3 Johnson's Algorithm (*TODO points*)

This is a **harder task** (it requires reading an algorithm from a research paper - it is recommended you return to this after trying Task 5 onwards to ensure you get as many marks as possible. Our current sparse multiplication algorithm may potentially generate many redundant intermediate polynomials. Even with Julia's native vectorisation over the provided `Heap` implementation, this can still become very slow. Implement [Johnson's Algorithm](#) ([this may also be useful](#)) for `PolynomialSparse`.

Use the existing multiplication tests to ensure your implementation is correct (you do not need to implement more tests).

For benchmarking, find a single instance where the existing implementation takes on average at least 5s to run. Demonstrate that your implementation is indeed an optimisation.

2.5 Task 5 (8pts) - Implementing $\mathbb{Z}_p$

We have provided an implementation of the *Cantor-Zassenhaus factoring algorithm* which allows for factoring polynomials when the coefficient ring is the field $\mathbb{Z}_p$. Previously, this course delved into the algorithm itself (and the corresponding algorithm to reconstruct a full factorisation over the integers). However, this was too involved (feel free to try to understand the algorithm/implementation if you wish to do so - but this is not necessary and will not be assessed) and instead we will simply utilise the implementation as a *blackbox*.

Until now, your polynomial implementation has assumed integer coefficients and (fatally) $\mathbb{Z}[x]$ is *not* a [euclidean domain](#). This means we do not yet have access to `gcd` on polynomials which is a critical component of the factoring algorithm. If we take our coefficients from a [field](#) like $\mathbb{Z}_p$, for p a prime, then the resulting $\mathbb{Z}_p[x]$ *is* a Euclidean domain. We therefore need to implement some way to work with $\mathbb{Z}_p[x]$.

Your task is to implement the coefficient field $\mathbb{Z}_p$, such that in the next task we can utilise it to factor polynomials. Throughout this section, you should create `_includes.jl` files as appropriate, and ensure you export the functions from the main module file.

Note: you should aim to implement all functions in this section as efficiently as you can. Additionally, if you can implement any of the following function in an equivalent (in particular, multiple dispatch should work with the same effectiveness) manner but a simpler type signature, you may do so.

TODO - add Base. to everything

2.5.1 Constructors (*2 points*)

1. Create folders `src/coefficients/z_mod_p/`. Inside of this folder, create a file `z_mod_p.jl` and create a `struct` with the following type signature:

```
struct ZModP{N} <: Integer
    val::Int # We will not implement this for BigInt, but in general val could be any subtype of Integer

    # val    [0,...,N - 1] - N is the prime p in Zp
```

end

Note: that N is a [value type](#) and thus it must be specified as a constant, it cannot be generated (e.g. via `rand`) at runtime. A collection of primes has been provided to you in `src/utls/sample_primes.jl`. Use subset of these to work with constants which are primes.

2. Create inner and outer constructors for this struct. You must be able to construct an object of type (TODO - `Val{N}?`) `ZModP{N}` where N is some prime. You must write functions satisfying *all* of the following type signatures:

```
function ZModP{N}(a::Int) where {N}
function ZModP(a::Int, ::Val{N})::ZModP{N} where {N}
function zero(::Type{ZModP{N}})::ZModP{N} where {N}
function one(::Type{ZModP{N}})::ZModP{N} where {N}
function zero(a::ZModP{N})::ZModP{N} where {N}
function one(a::ZModP{N})::ZModP{N} where {N}
```

3. Ensure the constructors in Task 5.2 (above) fail when attempting to create an object of type `ZModP{N}` when N is *not* a prime. Use the `Primes.jl` package that is included in the repository.

2.5.2 Field operations and miscellaneous functions (4 points)

4. Implement the following convenience and query operations:

```
function show(io::IO, a::ZModP{N}) where {N}
function ==(a::ZModP{N}, b::ZModP{N})::Bool where {N}
function ==(a::ZModP{N}, b::Integer)::Bool where {N}
function ==(a::Integer, b::ZModP{N})::Bool where {N}
```

5. Implement the following basic arithmetic (field) operations:

```
function +(a::ZModP{N}, b::ZModP{N}) where {N}
function +(a::ZModP{N}, b::Integer) where {N}
function +(a::Int, b::ZModP{N}) where {N}
function -(a::ZModP{N}) where {N}
function -(a::ZModP{N}, b::ZModP{N}) where {N}
function -(a::ZModP{N}, b::Integer) where {N}
function -(a::Integer, b::ZModP{N}) where {N}
function *(a::ZModP{N}, b::ZModP{N}) where {N}
function *(a::ZModP{N}, b::Integer) where {N}
function *(a::Integer, b::ZModP{N}) where {N}
function inv(a::ZModP{N})::ZModP{N} where {N}
function ÷(a::ZModP{N}, b::ZModP{N})::ZModP{N} where {N}
function ÷(a::ZModP{N}, b::Integer)::ZModP{N} where {N}
function ÷(a::Integer, b::ZModP{N})::ZModP{N} where {N}
```

Hint: Recall division in a field is simply multiplication by the inverse. See `int_inverse_mod` in `src/utls/general_alg.jl`.

Note: All the above operations should act as they do in $\mathbb{Z}_p$. E.g., $3 \cdot 2 = 6 \equiv 1 \pmod{5}$ should correspond to the code:

```
a = ZModP(3, Val(5))
@show a * 2 # should print 1
```

6. Implement the additional arithmetic operations

```
function ^(a::ZModP{N}, n::Integer) where {N}
function abs(a::ZModP{N})::ZModP{N} where {N}
```

7. Implement the following function to convert elements of `ZModP{N}` to `Int` or `BigInt`:

```
function (::Type{S})(a::ZModP)::S where {S <: Union{Int, BigInt}}
```

8. Override the function `div` for `Term` (i.e., in `src/term.jl`) to implement it for terms with coefficient type `ZModP` (the existing implementation should still be used for other coefficient fields).

Note: you may make as many additional convenience/helper functions as you wish.

2.5.3 Checking that it works (2 points)

10. Create `example_script_task_5.jl` and in this file, demonstrate *all* basic modular arithmetic operations working correctly (each at least once). TODO - Keep this and print output for PDF submission?
11. Create `tests/z_mod_p.jl` containing tests which run *all* operations on `ZModP`. Your tests must run on at least 10^3 randomly generated integers in the range $[-10^4, 10^4]$ inclusive and using at least 10 different primes. Utilise the in-built `mod` function on integers to verify correctness of your operations. Ensure your implementations pass these tests.
12. Add this test file to the `test/runtests.jl` file and have your tests print some output demonstrating the correctness of your operations on `ZModP`. *Screenshot (or copy/paste) the output of this and your example_script_task_5.jl and add it to your PDF submission.* TODO - edit this to be consistent with others (just one big test output and we can view tests ourselves)

2.6 Task 6 (7pts) - Using $\mathbb{Z}_p[x]$

2.6.1 Constructing $\mathbb{Z}_p[x]$ (5 points)

Now that we have `ZModP` as a subtype of `Number`, we can use this as the coefficient type of our polynomials. E.g., `PolynomialSparse{ZModP{Int, 5}}`. *Try experimenting in the REPL with this before attempting the rest of this section.*

Now that we have a working version of $\mathbb{Z}_p[x]$, we can use the provided factorisation algorithm! In particular, we can now work with **rational functions** (quotients of polynomials).

1. Check factorisation works with tests
2. Create a new folder

TODO - Make a quotient type, use it with polynomials of the same numerator/denominator type. Simplify via factorisation and then expansion. Have evaluation with errors on denominator equals 0. That's probably enough tbh.